Midterm Defense Presentation Script

Slide 1: Title Slide

> "Good afternoon, respected professors and judges. My name is Yujie Feng, and my supervisor is Professor Jin Hou. Today, I am honored to present my midterm progress on the project titled: 'AI-based Damage Detection for Power Grid Equipment with SaaS-oriented Design'."

Slide 2: Outline

> "My presentation today will **cover five main sections**. I will begin with the **project overview and a summary of my progress**. Then, I will dive into the **technical details of the three core detection modules**—Bird Nest, Oil Leak,

> and Infrared. After that, I will introduce my **SaaS Flask Platform**, and conclude with the challenges faced and the work plan for the next phase."

Slide 3: Project Overview: A Scalable SaaS Inspection Platform

> "To give you an overview, this project aims to replace risky and costly manual power equipment inspections with a scalable AI platform. As you can see, the platform consumes multi-modal inputs—ranging from RGB imagery to thermal images and telemetry. At the core, I have a SaaS backend with unified authentication, task isolated by tenant, and an AI model registry that currently hosts three modules. The ultimate enterprise outputs include alert dashboards and work order dispatch systems."

Slide 4: Midterm Progress Summary

> "My implementation strategy is to build a 'Vertical Slice First, then Horizontal Scale'. By midterm, I have successfully completed Module A for bird nest detection as an end-to-end Proof of Concept. Module B, for oil leaks, is currently fully trained on the dataset and deployed to the API. Lastly, Module C, handling infrared high-temperature detection, has been successfully implemented and integrated using a deep-learning-free approach."

Slide 5: Module A: Bird Nest Detection — Data & Model

> "Let's look at Module A. For data, we used 54 images with bounding box and instance polygon annotations. I specifically designed a deterministic augmentation pipeline, including a custom Random IoU crop, to avoid physically implausible artifacts that confuse the model. For the architecture, I selected Mask R-CNN with a ResNet50 backbone. Training utilized mixed precision (AMP), achieving about 27 seconds per epoch with excellent GPU utilization."

Slide 6: Module A: Validation Metrics

> "On the validation set, the model performed exceptionally well. The bounding box AP50 and segmentation AP50 both reached 1.0. [Point to the image] As you can see in the overlay result on the right, the instance segmentation mask precisely covers the bird nest, which is critical for calculating hazard zones in downstream work orders."

Slide 7: Module A: Training Dynamics

> "Looking at the training dynamics: the loss curves dropped from 1.41 to 0.14 without signs of overfitting, plateauing around epoch 20. The PR curves and IoU histograms demonstrate strong localization capability, even at strict IoU thresholds."

Slide 8: Module A: PR & Threshold Analysis

> "This slide shows the PR and F1 curves. Because the AP50 is perfect at 1.0, I found that a confidence threshold between 0.4 and 0.5 optimizes the F1-score for deployment, balancing missed detections with false positives."

Slide 9: Module B: Oil Leak Detection — Data & Training Config

Moving on to Oil Leak Detection.

The data strategy involves merging two sources: a **VOC-format dataset from the State Grid** project, and a **YOLO-format dataset collected from open-source platforms CSDN** . I wrote an automated script to unify these formats and labels into a consistent training set.

We train a YOLOv8-small model over 150 epochs using AdamW, Cosine learning rate, and mosaic augmentation.

Slide 10: Module B: Validation Metrics

> "The best validation metrics show a precision of 0.83 but a recall of 0.34, leading to an mAP50 (**在交并比(IoU) 阈值为0.50 时计算的平均精度均值**) of early 0.39.
>
> Current model achieves **0.83 precision but only 0.34 recall** —it's precise but conservative, missing many true positives.
>
> Next, we'll **enrich training data with more diverse leak patterns and calibrate the confidence threshold** to boost recall without sacrificing precision.

Slide 11: Module B: F1 & Confusion Matrix

> "Here's the normalized confusion matrix—it shows that while our precision is high, we're missing a significant portion of true positives, which aligns with the 0.34 recall we saw earlier. Currently, box-level detection is sufficient for our alerting system, but we can always upgrade to pixel-level segmentation later if the business requires precise area statistics."

Slide 12: Module C: Infrared High-Temperature Detection

"Module C handles thermal imaging using an OCR and pixel-mapping hybrid approach.

EasyOCR reads the temperature legend to establish a linear mapping between pixel intensity and actual temperature. The user's alarm threshold—say 50°C—is then converted into a pixel intensity threshold, and hotspots are isolated via grayscale thresholding.

To suppress false positives, I added two noise masks. The first masks out the **temperature scale bar** —located using the legend's position—to prevent its color gradient from being misdetected. The second filters out **white brand watermarks** using HSV color thresholding, regardless of their location.

如果老师问："你具体怎么做的mask？" 你就可以从容展开：

The mask actually serves two purposes:

First, it blocks the temperature scale bar—located using the legend's position—to prevent false positives from the color gradient.

Second, it suppresses white brand logos using HSV-based color filtering, regardless of their location in the image.

该mask实际上具有双重作用：

首先，它会遮挡温度刻度条（根据图例位置确定），以防止颜色渐变导致误判。

其次，它利用基于HSV的颜色过滤技术，无论品牌标识在图像中的位置如何，都能将其抑制。

Slide 13: Module C: Result

> "The outcome is a highly robust, training-free hotspot detector. [Point to figure] It successfully draws bounding boxes around the overheating component and overlays the exact delta temperature, while entirely ignoring the white legend text on the right side of the screen."

Slide 14: SaaS Flask Microservice — Architecture

[Action: Point to the TikZ diagram on screen]

> "All three of these modules are powered by a single SaaS Flask Microservice. As shown in this flowchart:
>
> 1. Clients push batch image uploads to the API Gateway.
> 2. The system checks the configured Model Registry and dynamically routes the image to the correct Inference Engine—whether it's Mask R-CNN, YOLO, or the OCR Hybrid.
> 3. Finally, it bundles the bounding boxes and visualized imagery into a JSON response. This architecture provides a scalable 'Model-as-a-Service' foundation."

Slide 15: SaaS Platform — User Interface

> "We wrapped this API into a user-friendly frontend. Users can log in, access the Main task Dashboard to select models and initiate batch inference, and then navigate to the History page to view analytical charts and download report ZIP files."

Slide 16: Problems & Challenges

> "During development, I've identified three main challenges: First, Generalization: perfect validation scores don't guarantee field success against haze or new towers. We need cross-scene data mining. Second, Technical choices: defining the boundary between building a functional AI demo versus a production-grade SaaS app with OAuth2. Third, Constraints: our dataset sizes are modest, making manual annotation a bottleneck."

Slide 17: Next Phase Work Plan

> "To address these, the next phase is split into two tracks: In Track 1 (Weeks 5-10), I will refine the SaaS backend with proper multi-tenant RBAC and productize our training utilities. In Track 2 (Weeks 11-18), I will complete the fully merged dataset training for the Oil Leak module. Finally, Weeks 19-20 are reserved for end-to-end multi-user testing and writing the thesis."

Slide 18: Discussion Points

> "That concludes my progress report. I look forward to your questions and would particularly welcome the committee's advice on prioritizing SaaS feature depth versus AI model generalization in the final phase. Thank you."